

Experiment 2 — Controlled cross-file defect injection

Master's thesis, Abdelrahman Khattab · Hasso Plattner Institute · run 2026-07-13 (design pre-registered 2026-07-05, before any code or data)

Why a second experiment

Experiment 1 showed KG context improves rubric scores on real PRs — but that evidence is *correlational* and *LLM-judged*. It cannot answer the question a committee will ask: **does the KG actually make the reviewer catch problems it would otherwise miss?** Experiment 2 answers it with a controlled, causal design: we plant defects with *known* cross-file consequences and measure whether each review mode detects them, against objective ground truth.

Design (pre-registered before implementation)

- **Defect taxonomy, 7 bands.** 5 *structural* bands whose consequence lives in a **different file** than the edit (S1 symbol rename, S2 signature change, S3 return-contract change, S4 base-class rename, S5 export removal) and 2 *local* control bands fully visible in the diff (L1 boundary-condition flip, L2 removed null/bounds check). The local bands are the placebo: if KG “wins” there too, its advantage would be a context-volume artefact, not structure.
- **40 injections** (28 structural + 12 local), discovered automatically and direction-blind (highest structural fan-out first, no review consulted) across **scikit-learn (Python)**, **kafka-streams (Java)**, **grafana-data (TypeScript)**. Each is presented as an innocuous refactor PR.
- **6 review setups compared:** baseline (diff-only) · **KG** (the deployed code-property-graph pipeline) · **RAG** (embedding retrieval, top-10) ·

hybrid (KG+RAG) · **KG + inheritance edges** (a pre-declared augmentation I built) · **KG idealised** (an upper bound using a perfect static dependency resolver).

- **Ground truth:** the true dependents of each edit, independently resolved by static analysis, with evidence lines recorded in a manifest.
- Generator and judge panel **identical to Experiment 1** (gpt-4o; 3-judge majority). Detection = the review names the planted break or a true off-diff dependent.
- **Decision thresholds fixed in advance** in the pre-registration, so the outcome could not be reframed after seeing data.

Results

Structural bands (n = 28) — detection rate [95% bootstrap CI]:

Setup	Detected	Rate
baseline	0/28	0.00
RAG	1/28	0.04 [0.00, 0.11]
hybrid	12/28	0.43 [0.25, 0.61]
KG (deployed)	15/28	0.54 [0.36, 0.71]
KG + inheritance edges	21/28	0.75 [0.57, 0.89]
KG idealised (upper bound)	26/28	0.93 [0.82, 1.00]

KG – baseline = +0.54, $p < 0.0001$ (paired permutation test) — the pre-registered success threshold ($\geq +0.30$) is met decisively. RAG – baseline = +0.04, $p = 1.0$.

Local control bands (n = 12): KG – baseline = -0.08 , inside the pre-registered parity window $[-0.15, +0.15]$. All setups detect 83–100% of local defects (they are visible in the diff, as designed). **The KG advantage is structural, not “more context is always better”.**

Base-class rename band (n = 6): the standard graph tool misses inheritance relationships; adding import/inheritance edges lifts detection from 2/6 to 5/6 (meets the pre-registered $\geq +0.30$ threshold) — a concrete engineering contribution of the thesis.

Judge validity: each judge was checked against a deterministic text-matching oracle (does the review literally mention a true dependent?): 72–93% agreement, and **zero hallucinated detections** for both OpenAI judges.

Why RAG fails here (one paragraph)

Similarity retrieval answers “what looks like this code?”, not “what depends on this code?”. The true dependents of a renamed symbol are usage sites — often boilerplate-unlike, semantically distant files that embedding similarity never surfaces. A structure-aware variant confirms this is architectural, not an implementation bug: giving the same retrieval pipeline dependency edges instead of similarity recovers detection. (A detailed example-by-example analysis is available.)

Honest notes

- 40 injections instead of the pre-registered 48: some defect types don’t exist in every language (e.g. Java has no export “barrel” files), and the shortfall is reported per band rather than back-filled — a rule fixed in the pre-registration.
- Java is the deployed KG’s weakest repo (2/8): having the right edge in the graph is necessary but not sufficient — how prominently it is presented to the model matters too.
- One TypeScript codebase substituted for the pre-registered Go one (the CPG tool has no Go support); documented before the run.
- Defects are synthetic by construction — that is the price of ground truth. External validity comes from Experiment 1 (real PRs), and human validity from the accompanying rater study.

How the pieces fit

Evidence	Shows	Limitation it leaves
Experiment 1 (40 real PRs, LLM judges)	KG improves the criteria it targets at scale	machine-judged, correlational
Experiment 2 (this doc)	KG context is <i>causally</i> the difference between detecting and missing a cross-file defect	synthetic defects, no humans
Human study (live now)	blinded humans perceive the same specific	perception probe, small n

Evidence	Shows	Limitation it leaves
	difference (grounding of affected components)	

All three converge on one construct: **knowing the true dependents of a change.**